



School of Electronics Engineering (SENSE)

Department of Electronics and Communication

Engineering

[2025-2026]

PROJECT REPORT on

VibePiano Glove: Gesture-Based Air Piano System

Submitted by :

Team Members:

1. Phanindra [24BEC0501]
2. Balaji [24BEC0618]

VibePiano Glove: Gesture-Based Air Piano System

ABSTRACT:

The VibePiano Glove is an innovative wearable device combining five flex sensors for individual finger note detection (C4-G4) with MPU6050 IMU for wrist motion control. Flex sensors on each finger (thumb→pinky) detect precise bends processed by ESP32 microcontroller, while MPU6050 accelerometer and gyroscope enable dynamic performance controls including volume adjustment ($\pm 90^\circ$ wrist roll), octave shifting (palm orientation).

Vibration motors provide proportional haptic feedback simulating real piano key press, with intensity varying by bend angle. ESP32 processes multi-modal inputs (flex + motion) with $< 25\text{ms}$ latency, transmitting note data and gesture parameters via Bluetooth for real-time audio synthesis on connected devices. MPU6050 integration transforms simple finger triggers into expressive performance instrument.

This cost-effective system demonstrates reliable finger-note mapping, smooth motion gesture recognition, and intuitive haptic interaction. Results validate wearable technology's potential for professional musical applications, combining finger precision with hand expressivity for complete air piano experience.

Literature Survey:

Recent studies in wearable technology focus on smart gloves for gesture recognition using flex sensors. Air piano systems using flex sensors and computer vision have shown good accuracy, but flex-based methods suffer from higher latency. Research also highlights that proper placement of flex sensors improves accuracy and signal stability. Smart gloves using multiple sensors can convert gestures into text or speech for communication. Advanced sensors and machine learning improve performance but increase complexity. Haptic feedback using vibration motors enhances user interaction, making wearable systems more intuitive and effective.

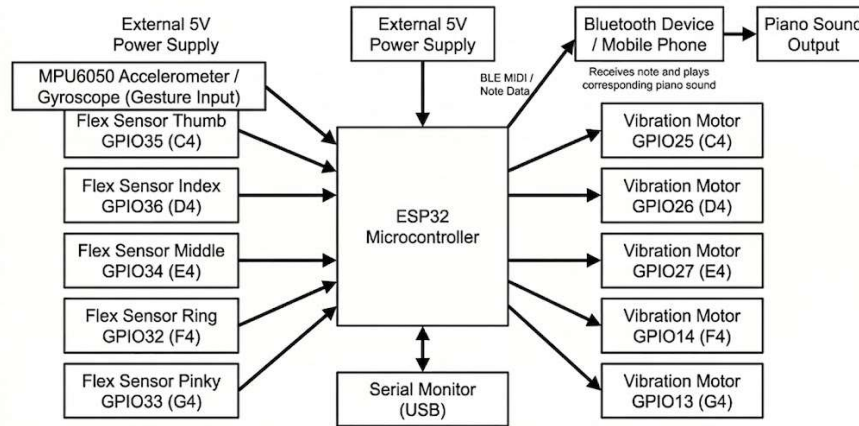
Year	Title of the paper	Methodology/ Approach	Key Findings	Limitations
2024	Air Piano Interaction via Flex Sensors and Computer Vision on Edge Devices.	The research explores two creative ways to bring an invisible piano to life using portable technology. The first method is a "wearable" approach, where you slip on a glove equipped with flex sensors that translate your finger bends into musical notes via an Arduino UNO. It even uses an ultrasonic sensor to let you change the pitch just by moving your hand through the air, much like a theremin. While this glove is incredibly power-efficient—using almost no energy—it can be a bit slow to respond and less precise. Alternatively, the "contactless" method uses a camera and an AI-powered Jetson Nano to "see" your hands and map your gestures to music. This version is much faster and more accurate, though it requires significantly more power to run the complex vision software.	The study highlights a fascinating trade-off between "seeing" music and "feeling" it through different technologies. The Computer Vision method is the high performer, offering a smooth, responsive experience with 95.7% accuracy and a quick 134ms response time. However, this precision comes at a cost, as the Jetson Nano processor is quite power-hungry. On the other hand, the Flex Sensor glove is incredibly efficient, using almost no power (0.25 mW), making it perfect for portable use. While the glove is a bit slower and less precise due to sensitive sensor data, it remains a highly practical, battery-friendly way to play.	The Flex Sensor method's main drawback is its unstable sensor data, leading to a lower accuracy of 83.15% and a sluggish 505 ms response time. Conversely, the Computer Vision approach is hindered by high power consumption and hardware constraints. Running Mediapipe on the Jetson Nano is limited by its 4 GB RAM and 1.43 GHz processor, and since it lacks PWM pins, an extra Arduino is required for audio.

2021	Design and Implementation of an Angry Birds Game Controller Using Flex Sensor and a Potentiometer.	This project transforms a wearable interface into a specialized gaming controller using an Arduino Nano. It maps the resistance changes of a flex sensor to the "pulling" tension of a slingshot and uses a potentiometer to calculate the vertical launch angle.	The custom controller successfully replaced traditional mouse inputs, providing a more tactile and immersive experience. It proved that simple analog components can effectively translate physical force into software-based velocity and trajectory physics.	The device is currently wired, which can restrict the user's natural range of motion during gameplay. Additionally, the system's accuracy is highly dependent on the initial calibration of the sensor's resting resistance.
2021	Development of a Low-cost Data Glove using Flex Sensors for the Robot Hand Teleoperation	This system uses five flex sensors to control a 5-DOF robotic hand via master-slave architecture. An Arduino Mega processes the hand movements and transmits data wirelessly using HC-05 Bluetooth modules to a receiver that drives the robotic servos.	The glove achieved a high success rate in remote grasping and releasing tasks. The wireless link maintained stable communication with minimal lag, offering a viable, low-cost solution for handling materials in hazardous environments.	The robotic hand is limited to basic flexion and extension and lacks haptic feedback, meaning the operator cannot perceive the pressure or grip force being applied to an object.
2021	Modelling of Sign Language Smart Glove Based on Bit Equivalent Implementation Using Flex Sensor	This research introduces a Bit-Equivalent logic model to translate flex sensor data into the American Sign Language (ASL) alphabet. It utilizes an Arduino Nano to process the "bits" of data and displays the resulting characters on a 16x2 LCD screen	By simplifying complex sensor data into binary-like "bit" states, the system significantly reduced computational overhead while maintaining high recognition accuracy. This allows for faster real-time translation on low-power microcontrollers	The model primarily identifies static letters and struggles to distinguish between gestures that share similar finger orientations but require different hand movements or orientations.

2025	Investigation of the Effect of the Location of Flex Sensors in a Sensor Glove for Accurate Tracking of Hand Movements	This research focuses on the "wearable" hardware side of motion capture. Researchers tested four different placement configurations on a subject's index finger: along the phalanx, at the nail base, on the dorsal surface, and on the lateral surface. The system used a FLEX 2.2 model strip, an STM32 microcontroller, and a digital filtering algorithm to process finger flexion into real-time angle data.	The study found that location is everything when it comes to accuracy. The most reliable data—with minimal error and high repeatability—occurred when the sensor was centered directly over the finger joints. Placing sensors too far from joints caused "smoothed" graphs and underestimated the actual bending values. Additionally, using a buffer amplifier and digital filtering was found to be essential for stabilizing the raw resistive signals against mechanical noise.	A major drawback is that standard flex sensors are "one-dimensional"—they can track bending (flexion) but are completely unsuitable for lateral movements like finger abduction. The sensors are also prone to errors of up to 10% caused by physical wear and tear or environmental temperature changes. Furthermore, excessive fastener tension or a poor fit against the skin can cause sharp signal fluctuations or even structural damage to the sensor substrate.
2022	Design and Validation of Data Glove with Flex Sensor and 9-Axis IMU to Assess Hand Dexterity	This clinical-grade glove combines five flex sensors with a 9-axis BNO055 IMU for comprehensive hand tracking. A Raspberry Pi Pico logs high-resolution kinematic data, capturing both fine finger joint angles and overall palm orientation in 3D space. The system was validated in a Unity 3D environment to drive a digital hand model in real-time.	The dual-sensor approach was successfully validated against standard clinical metrics for assessing hand dexterity and motor functionality. It provides a reliable, objective tool for tracking patient progress in medical rehabilitation, specifically for stroke victims. The high-precision IMU data ensures that the palm's spatial orientation is correctly mapped..	The glove's wired USB connection may slightly interfere with a patient's natural movement during testing, potentially skewing dexterity results. Furthermore, the inclusion of a high-precision 9-axis IMU and specialized microcontrollers increases the overall production cost compared to simpler models.

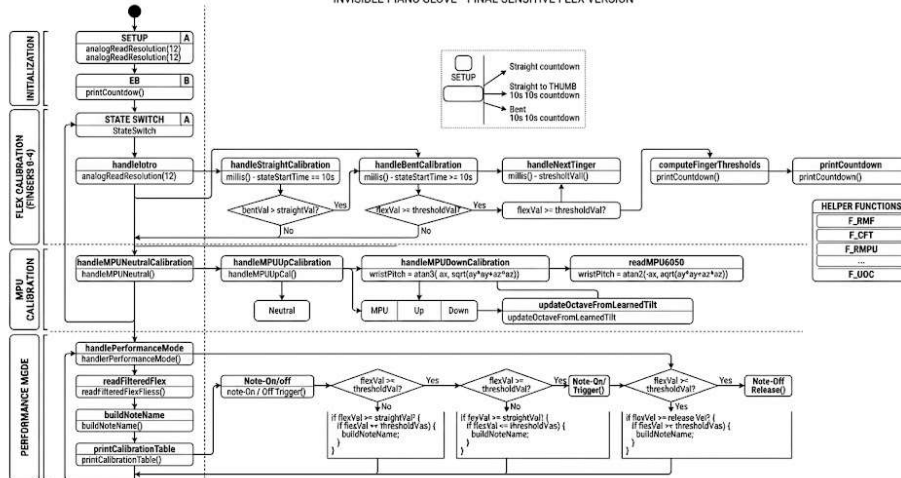
2024	Machine Learning Enhanced Smart Sensors for Hand Gesture Recognition	This study proposes an intelligent sensor system that combines Flex sensors, JY901S motion sensors, and a camera. It uses Support Vector Machines (SVM) for static gestures and Long Short-Term Memory (LSTM) networks for dynamic gestures. Data is collected via an Arduino MEGA and Google's MediaPipe library for hand landmark detection.	Integrating multi-modal data significantly boosts accuracy compared to single-sensor setups. Static gesture models achieved up to 99.96% accuracy using SVM, while dynamic gestures reached 100% accuracy with the LSTM model. Data augmentation with Gaussian noise helped expand the dataset to 180,000 samples for more robust training.	Real-time data processing remains a challenge. The system also struggles with the complexity of fusing multi-modal data and ensuring that algorithms generalize well across different complex application scenarios.
2024	Design of Multichannel Tactile Feedback Digital Glove for Virtual Reality	This project focuses on immersion by adding vibration and thermal tactile feedback to a data glove. It uses an ESP32 core, strain gauge sensors for finger posture, and carbon fiber heating pads for temperature simulation. The system was tested in a Unity-base	The glove significantly improved the user's sense of "realism" and "experience" compared to standard VR controllers. Using PWM (Pulse Width Modulation), researchers established three distinct levels of haptic intensity—first-stage, medium, and strong. Users reported a more intuitive experience since they didn't have to keep buttons pressed to simulate clenching.	The system is physically large and heavy, which can reduce comfort. Hand positioning still relies on a separate VR controller. Additionally, there is a lag in temperature response because an open-loop control strategy is used for the heating pads.

Methodology:



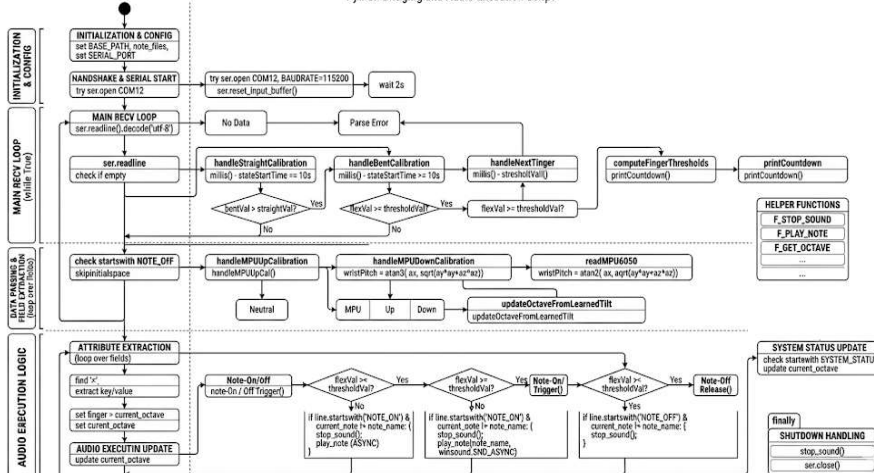
INVISIBLE PIANO GLOVE - FIRMWARE LOGIC FLOW (ESP32)

INVISIBLE PIANO GLOVE - FINAL SENSITIVE FLEX VERSION



INVISIBLE PIANO PC SOUND ENGINE - SOFTWARE LOGIC FLOW (Python)

Python Bridging and Audio Execution Script



Software Code

Language: C++ | Platform: Arduino IDE + ESP32 board

```
#include<Wire.h>

#include<Adafruit_MPU6050.h>

#include<Adafruit_Sensor.h>

#include<math.h>

const int NUM_FINGERS=5;

const int flexPins[NUM_FINGERS]={35,36,34,32,33};

const int motorPins[NUM_FINGERS]={25,26,27,14,13};

const char*fingerNames[NUM_FINGERS]={"THUMB","INDEX","MIDDLE","RING","PINKY"};

const char*baseNotes[NUM_FINGERS]={"C","D","E","F","G"};

int straightVal[NUM_FINGERS]={0,0,0,0,0};

int bentVal[NUM_FINGERS]={0,0,0,0,0};

int thresholdVal[NUM_FINGERS]={0,0,0,0,0};

int releaseVal[NUM_FINGERS]={0,0,0,0,0};

int bendRangeVal[NUM_FINGERS]={0,0,0,0,0};

bool noteOn[NUM_FINGERS]={false,false,false,false,false};

const unsigned long captureDuration=10000;

const unsigned long mpuReadInterval=40;

const float pressPercent=0.15;

const float releasePercent=0.08;

const int minimumRange=60;

const int adcFilterWeightNew=35;

unsigned long stateStartTime=0,lastStatusPrint=0,lastMpuRead=0;

int currentFinger=0;

bool motorsEnabled=false;

float filteredFlex[NUM_FINGERS]={0,0,0,0,0};

Adafruit_MPU6050 mpu;

float wristPitch=0.0,wristRoll=0.0;

float neutralPitch=0.0,upPitch=0.0,downPitch=0.0;

int currentOctave=4;

enum SystemState{SHOW_INTRO,CALIBRATE_STRAIGHT,CALIBRATE_BENT,NEXT_FINGER,CALIBRATE_MPU_NEUTRAL,CALIBRATE_MPU_UP,CALIBRATE_MPU_DOWN,CALIBRATION_DONE,PERFORMANCE_MODE};

SystemState systemState=SHOW_INTRO;

void printIntro();

void handleIntro();

void handleStraightCalibration();

void handleBentCalibration();

void handleNextFinger();

void handleMPUNeutralCalibration();

void handleMPUUpCalibration();

void handleMPUDownCalibration();

void handleCalibrationDone();

void handlePerformanceModel();
```

```

void printCountdown(int,const char*);

void readMPU6050();

void updateOctaveFromLearnedTilt();

String buildNoteName(int,int);

void printCalibrationTable();

int readFilteredFlex(int);

void computeFingerThresholds(int);

void setup(){
  Serial.begin(115200);

  delay(1500);

  for(int i=0;i<NUM_FINGERS;i++){pinMode(motorPins[i],OUTPUT);digitalWrite(motorPins[i],LOW);filteredFlex[i]=analogRead(flexPins[i]);}

  Wire.begin(21,22);

  if(!mpu.begin()){Serial.println("MPU6050 NOT DETECTED. CHECK WIRING.");while(1)delay(10);}

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  printIntro();

  stateStartTime=millis();
}

void loop(){
  switch(systemState){

    case SHOW_INTRO:handleIntro();break;

    case CALIBRATE_STRAIGHT:handleStraightCalibration();break;

    case CALIBRATE_BENT:handleBentCalibration();break;

    case NEXT_FINGER:handleNextFinger();break;

    case CALIBRATE_MPU_NEUTRAL:handleMPUNeutralCalibration();break;

    case CALIBRATE_MPU_UP:handleMPUUpCalibration();break;

    case CALIBRATE_MPU_DOWN:handleMPUDownCalibration();break;

    case CALIBRATION_DONE:handleCalibrationDone();break;

    case PERFORMANCE_MODE:handlePerformanceMode();break;

  }}

void printIntro(){
  Serial.println();

  Serial.println("=====");

  Serial.println("INVISIBLE PIANO GLOVE - FINAL SENSITIVE VERSION");

  Serial.println("=====");

  Serial.println("STEP 1: FLEX CALIBRATION");

  Serial.println("STEP 2: MPU CALIBRATION");

  Serial.println("NEUTRAL = OCTAVE 4");

  Serial.println("UP TILT = OCTAVE 5");

  Serial.println("DOWN TILT = OCTAVE 3");

  Serial.println("=====");

}

void handleIntro(){
  Serial.println("STARTING FINGER CALIBRATION...");

```

```

Serial.println("FIRST FINGER: THUMB");

delay(1500);

currentFinger=0;

stateStartTime=millis();

lastStatusPrint=0;

systemState=CALIBRATE_STRAIGHT;

}

void handleStraightCalibration(){

static long sum=0;static int count=0;static bool initialized=false;

if(!initialized){sum=0;count=0;initialized=true;Serial.print("KEEP ");Serial.print(fingerNames[currentFinger]);Serial.println(" STRAIGHT FOR 10 SECONDS...");}

int flexVal=analogRead(flexPins[currentFinger]);

sum+=flexVal;count++;

printCountdown(flexVal,"STRAIGHT");

if(millis()-stateStartTime>=captureDuration){

straightVal[currentFinger]=sum/count;

Serial.print("STRAIGHT SAVED FOR ");Serial.print(fingerNames[currentFinger]);Serial.print(": ");Serial.println(straightVal[currentFinger]);

initialized=false;stateStartTime=millis();lastStatusPrint=0;systemState=CALIBRATE_BENT;}

}

void handleBentCalibration(){

static long sum=0;static int count=0;static bool initialized=false;

if(!initialized){sum=0;count=0;initialized=true;Serial.print("BEND ");Serial.print(fingerNames[currentFinger]);Serial.println(" FULLY FOR 10 SECONDS...");}

int flexVal=analogRead(flexPins[currentFinger]);

sum+=flexVal;count++;

printCountdown(flexVal,"BENT");

if(millis()-stateStartTime>=captureDuration){

bentVal[currentFinger]=sum/count;

computeFingerThresholds(currentFinger);

Serial.print("BENT SAVED FOR ");Serial.print(fingerNames[currentFinger]);Serial.print(": ");Serial.println(bentVal[currentFinger]);

Serial.print("RANGE FOR ");Serial.print(fingerNames[currentFinger]);Serial.print(": ");Serial.println(bendRangeVal[currentFinger]);

Serial.print("PRESS THRESHOLD FOR ");Serial.print(fingerNames[currentFinger]);Serial.print(": ");Serial.println(thresholdVal[currentFinger]);

Serial.print("RELEASE THRESHOLD FOR ");Serial.print(fingerNames[currentFinger]);Serial.print(": ");Serial.println(releaseVal[currentFinger]);

initialized=false;stateStartTime=millis();systemState=NEXT_FINGER;}

}

void handleNextFinger(){

currentFinger++;

if(currentFinger<NUM_FINGERS){

Serial.println("-----");

Serial.print("NEXT FINGER: ");Serial.println(fingerNames[currentFinger]);

Serial.println("-----");

delay(1200);stateStartTime=millis();lastStatusPrint=0;systemState=CALIBRATE_STRAIGHT;

}else{

Serial.println("ALL FLEX SENSORS CALIBRATED.");

Serial.println("STARTING MPU CALIBRATION...");

delay(1500);stateStartTime=millis();lastStatusPrint=0;systemState=CALIBRATE_MPU_NEUTRAL;}

}

```

```

void handleMPUNeutralCalibration(){

static float sum=0;static int count=0;static bool initialized=false;

if(!initialized){sum=0;count=0;initialized=true;Serial.println("KEEP HAND IN NEUTRAL POSITION FOR 10 SECONDS...");}

readMPU6050();sum+=wristPitch;count++;

if((millis()-lastStatusPrint>=1000){lastStatusPrint=millis();Serial.print("NEUTRAL PITCH = ");Serial.println(wristPitch,1);}

if((millis()-stateStartTime>=captureDuration){

neutralPitch=sum/count;

Serial.print("NEUTRAL SAVED: ");Serial.println(neutralPitch,2);

initialized=false;stateStartTime=millis();lastStatusPrint=0;systemState=CALIBRATE_MPU_UP;}

}

void handleMPUUpCalibration(){

static float sum=0;static int count=0;static bool initialized=false;

if(!initialized){sum=0;count=0;initialized=true;Serial.println("TILT HAND UP FOR 10 SECONDS...");}

readMPU6050();sum+=wristPitch;count++;

if((millis()-lastStatusPrint>=1000){lastStatusPrint=millis();Serial.print("UP PITCH = ");Serial.println(wristPitch,1);}

if((millis()-stateStartTime>=captureDuration){

upPitch=sum/count;

Serial.print("UP SAVED: ");Serial.println(upPitch,2);

initialized=false;stateStartTime=millis();lastStatusPrint=0;systemState=CALIBRATE_MPU_DOWN;}

}

void handleMPUDownCalibration(){

static float sum=0;static int count=0;static bool initialized=false;

if(!initialized){sum=0;count=0;initialized=true;Serial.println("TILT HAND DOWN FOR 10 SECONDS...");}

readMPU6050();sum+=wristPitch;count++;

if((millis()-lastStatusPrint>=1000){lastStatusPrint=millis();Serial.print("DOWN PITCH = ");Serial.println(wristPitch,1);}

if((millis()-stateStartTime>=captureDuration){

downPitch=sum/count;

Serial.print("DOWN SAVED: ");Serial.println(downPitch,2);

initialized=false;systemState=CALIBRATION_DONE;}

}

void handleCalibrationDone(){

Serial.println("=====");

Serial.println("CALIBRATION COMPLETED SUCCESSFULLY");

printCalibrationTable();

Serial.println("SYSTEM READY");

Serial.println("=====");

delay(2000);

systemState=PERFORMANCE_MODE;

}

```

Language: Python 3 | Platform: Windows + pyserial library

```
import serial

import time

import winsound

import os

import csv

SERIAL_PORT = "COM12"

BAUD_RATE = 115200

BASE_PATH = r"C:\Users\ASUS\Desktop\piano"

note_files = {

    "C3": os.path.join(BASE_PATH, "C3.wav"),

    "D3": os.path.join(BASE_PATH, "D3.wav"),

    "E3": os.path.join(BASE_PATH, "E3.wav"),

    "F3": os.path.join(BASE_PATH, "F3.wav"),

    "G3": os.path.join(BASE_PATH, "G3.wav"),

    "C4": os.path.join(BASE_PATH, "C4.wav"),

    "D4": os.path.join(BASE_PATH, "D4.wav"),

    "E4": os.path.join(BASE_PATH, "E4.wav"),

    "F4": os.path.join(BASE_PATH, "F4.wav"),

    "G4": os.path.join(BASE_PATH, "G4.wav"),

    "C5": os.path.join(BASE_PATH, "C5.wav"),

    "D5": os.path.join(BASE_PATH, "D5.wav"),

    "E5": os.path.join(BASE_PATH, "E5.wav"),

    "F5": os.path.join(BASE_PATH, "F5.wav"),

    "G5": os.path.join(BASE_PATH, "G5.wav")

}

missing = [note for note, path in note_files.items() if not os.path.exists(path)]

if missing:

    print("Missing WAV files:", " ", " ".join(missing))

    raise SystemExit

try:

    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

    time.sleep(2)

    ser.reset_input_buffer()

    print("Connected to", SERIAL_PORT)

except Exception as e:

    print("Serial error:", e)

    raise SystemExit

current_note = None
```

```

current_octave = 4

def stop_sound():
    winsound.PlaySound(None, 0)

def play_note(note_name):
    winsound.PlaySound(
        note_files[note_name],
        winsound.SND_FILENAME | winsound.SND_ASYNC
    )

try:
    while True:
        line = ser.readline().decode("utf-8", errors="ignore").strip()

        if not line:
            continue

        print(line)

        if line.startswith("NOTE_ON,") or line.startswith("NOTE_OFF,"):
            try:
                fields = next(csv.reader([line], skipinitialspace=True))

                note_name = fields[1]

                finger = "UNKNOWN"

                for field in fields[2:]:
                    if "=" in field:
                        key, value = field.split("=", 1)

                        if key == "FINGER":
                            finger = value

                        elif key == "OCTAVE":
                            try:
                                current_octave = int(value)
                            except:
                                pass

                if note_name not in note_files:
                    print("Unknown note:", note_name)

                    continue

                if line.startswith("NOTE_ON,"):
                    if current_note != note_name:
                        stop_sound()

                        play_note(note_name)

                        current_note = note_name

                    print(f"PLAYING {note_name} | Finger={finger} | Octave={current_octave}")

                elif line.startswith("NOTE_OFF,"):
                    if current_note == note_name:
                        stop_sound()

```

```

        print(f"STOPPED {note_name} | Finger={finger} | Octave={current_octave}")

        current_note = None

    except Exception as e:

        print("Parse error:", e)

    elif line.startswith("SYSTEM_STATUS"):

        try:

            if "OCTAVE=" in line:

                current_octave = int((line.split("OCTAVE=")[1].split(",")[0]))

                print("CURRENT OCTAVE =", current_octave)

            except:

                pass

except KeyboardInterrupt:

    print("Stopped by user")

finally:

    stop_sound()

    ser.close()

    print("Serial closed")

```

Results:

Calibration values of middle finger:

```

=====
NEXT FINGER: MIDDLE
=====
KEEP MIDDLE STRAIGHT FOR 10 SECONDS...
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=133,REMAINING=10s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=137,REMAINING=9s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=138,REMAINING=8s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=135,REMAINING=7s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=138,REMAINING=6s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=138,REMAINING=5s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=129,REMAINING=4s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=132,REMAINING=3s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=135,REMAINING=2s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=132,REMAINING=1s
FINGER=MIDDLE,MODE=STRAIGHT,VALUE=134,REMAINING=0s
STRAIGHT SAVED FOR MIDDLE: 134
BEND MIDDLE FULLY FOR 10 SECONDS...
FINGER=MIDDLE,MODE=BENT,VALUE=133,REMAINING=10s
FINGER=MIDDLE,MODE=BENT,VALUE=133,REMAINING=9s
FINGER=MIDDLE,MODE=BENT,VALUE=80,REMAINING=8s
FINGER=MIDDLE,MODE=BENT,VALUE=115,REMAINING=7s
FINGER=MIDDLE,MODE=BENT,VALUE=62,REMAINING=6s
FINGER=MIDDLE,MODE=BENT,VALUE=66,REMAINING=5s
FINGER=MIDDLE,MODE=BENT,VALUE=55,REMAINING=4s
FINGER=MIDDLE,MODE=BENT,VALUE=117,REMAINING=3s
FINGER=MIDDLE,MODE=BENT,VALUE=98,REMAINING=2s
FINGER=MIDDLE,MODE=BENT,VALUE=117,REMAINING=1s
FINGER=MIDDLE,MODE=BENT,VALUE=107,REMAINING=0s
BENT SAVED FOR MIDDLE: 98
RANGE FOR MIDDLE: 60
PRESS THRESHOLD FOR MIDDLE: 125
RELEASE THRESHOLD FOR MIDDLE: 130
=====

```

Calibration of MPU6050 to shift between different octaves:

```
Command Prompt - py invisib
RELEASE THRESHOLD FOR PINKY: 148
ALL FLEX SENSORS CALIBRATED.
STARTING MPU CALIBRATION.
KEEP HAND IN NEUTRAL POSITION FOR 10 SECONDS...
NEUTRAL PITCH = 3.5
NEUTRAL PITCH = 3.4
NEUTRAL PITCH = 3.3
NEUTRAL PITCH = 3.3
NEUTRAL PITCH = 3.2
NEUTRAL PITCH = 3.3
NEUTRAL PITCH = 3.3
NEUTRAL PITCH = 3.5
NEUTRAL PITCH = 3.7
NEUTRAL PITCH = 2.4
NEUTRAL SAVED: 3.68
TILT HAND UP FOR 10 SECONDS...
UP PITCH = 5.8
UP PITCH = 5.1
UP PITCH = -3.7
UP PITCH = -46.7
UP PITCH = -52.4
UP PITCH = -58.2
UP PITCH = -80.6
UP PITCH = -78.3
UP PITCH = -53.6
UP PITCH = -55.5
UP PITCH = -55.2
UP SAVED: -46.23
TILT HAND DOWN FOR 10 SECONDS...
DOWN PITCH = -55.2
DOWN PITCH = -15.6
DOWN PITCH = 10.3
DOWN PITCH = 43.1
DOWN PITCH = 60.7
DOWN PITCH = 57.9
DOWN PITCH = 63.1
DOWN PITCH = 62.6
DOWN PITCH = 62.0
DOWN PITCH = 58.5
DOWN SAVED: 40.76
=====
CALIBRATION COMPLETED SUCCESSFULLY
Finger | Straight | Bent | Range | Press | Release
-----
THUMB | 0 | 0 | 60 | -9 | -4
INDEX | 3529 | 925 | 2984 | 3394 | 3597
MIDDLE | 134 | 98 | 60 | 125 | 130
RING | 0 | 0 | 60 | -9 | -4
PINKY | 152 | 119 | 60 | 143 | 148
=====
```

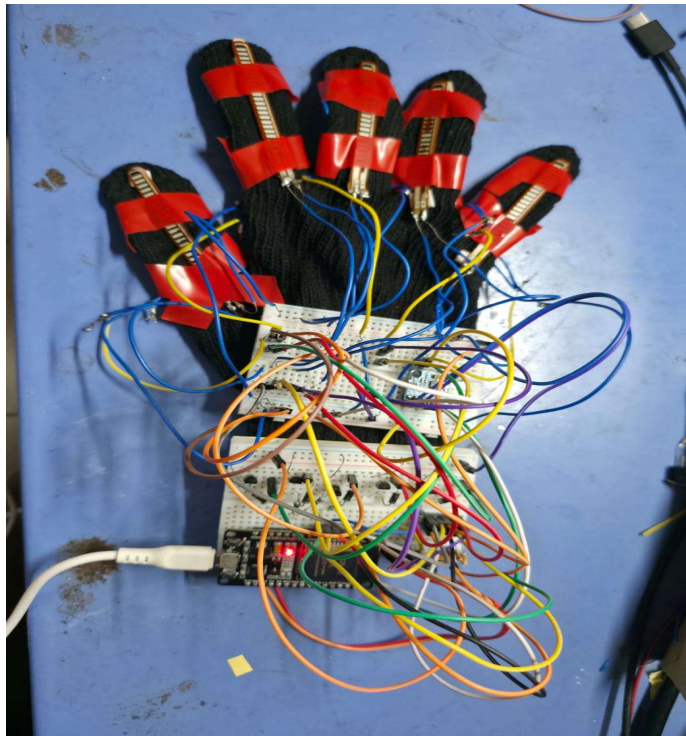
Data set Values for mpu and flex after calibration:

```
Command Prompt - py invisib
=====
NOTE_ON, D4, FINGER=INDEX, FLEX=21, PRESS=3394, RELEASE=3597, OCTAVE=4
PLAYING D4 | FINGER=INDEX | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=4.5, F0=0, F1=35, F2=123, F3=0, F4=143
CURRENT OCTAVE = 4
NOTE_ON, E4, FINGER=MIDDLE, FLEX=120, PRESS=125, RELEASE=130, OCTAVE=4
PLAYING E4 | FINGER=MIDDLE | Octave=4
NOTE_ON, G4, FINGER=PINKY, FLEX=140, PRESS=143, RELEASE=148, OCTAVE=4
PLAYING G4 | FINGER=PINKY | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.6, F0=0, F1=60, F2=114, F3=0, F4=130
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.6, F0=0, F1=59, F2=113, F3=0, F4=133
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.8, F0=0, F1=60, F2=115, F3=0, F4=134
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.8, F0=0, F1=61, F2=114, F3=0, F4=133
CURRENT OCTAVE = 4
NOTE_OFF, G4, FINGER=PINKY, FLEX=149, OCTAVE=4
STOPPED G4 | FINGER=PINKY | Octave=4
NOTE_ON, G4, FINGER=PINKY, FLEX=141, PRESS=143, RELEASE=148, OCTAVE=4
PLAYING G4 | FINGER=PINKY | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=2.6, F0=0, F1=48, F2=113, F3=0, F4=131
CURRENT OCTAVE = 4
NOTE_OFF, D4, FINGER=INDEX, FLEX=3630, OCTAVE=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.7, F0=0, F1=4094, F2=113, F3=0, F4=134
CURRENT OCTAVE = 4
NOTE_ON, D4, FINGER=INDEX, FLEX=2680, PRESS=3394, RELEASE=3597, OCTAVE=4
PLAYING D4 | FINGER=INDEX | Octave=4
NOTE_OFF, G4, FINGER=PINKY, FLEX=150, OCTAVE=4
NOTE_ON, G4, FINGER=PINKY, FLEX=141, PRESS=143, RELEASE=148, OCTAVE=4
PLAYING G4 | FINGER=PINKY | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=2.5, F0=0, F1=58, F2=97, F3=0, F4=135
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.0, F0=0, F1=58, F2=111, F3=0, F4=134
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.5, F0=0, F1=60, F2=111, F3=0, F4=136
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=3.7, F0=0, F1=55, F2=112, F3=0, F4=135
CURRENT OCTAVE = 4
NOTE_OFF, G4, FINGER=PINKY, FLEX=149, OCTAVE=4
STOPPED G4 | FINGER=PINKY | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=2.9, F0=0, F1=58, F2=112, F3=0, F4=150
CURRENT OCTAVE = 4
NOTE_ON, G4, FINGER=PINKY, FLEX=139, PRESS=143, RELEASE=148, OCTAVE=4
PLAYING G4 | FINGER=PINKY | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=4.2, F0=0, F1=58, F2=114, F3=0, F4=140
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=4.0, F0=0, F1=78, F2=112, F3=0, F4=133
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=4.2, F0=0, F1=48, F2=118, F3=0, F4=136
```

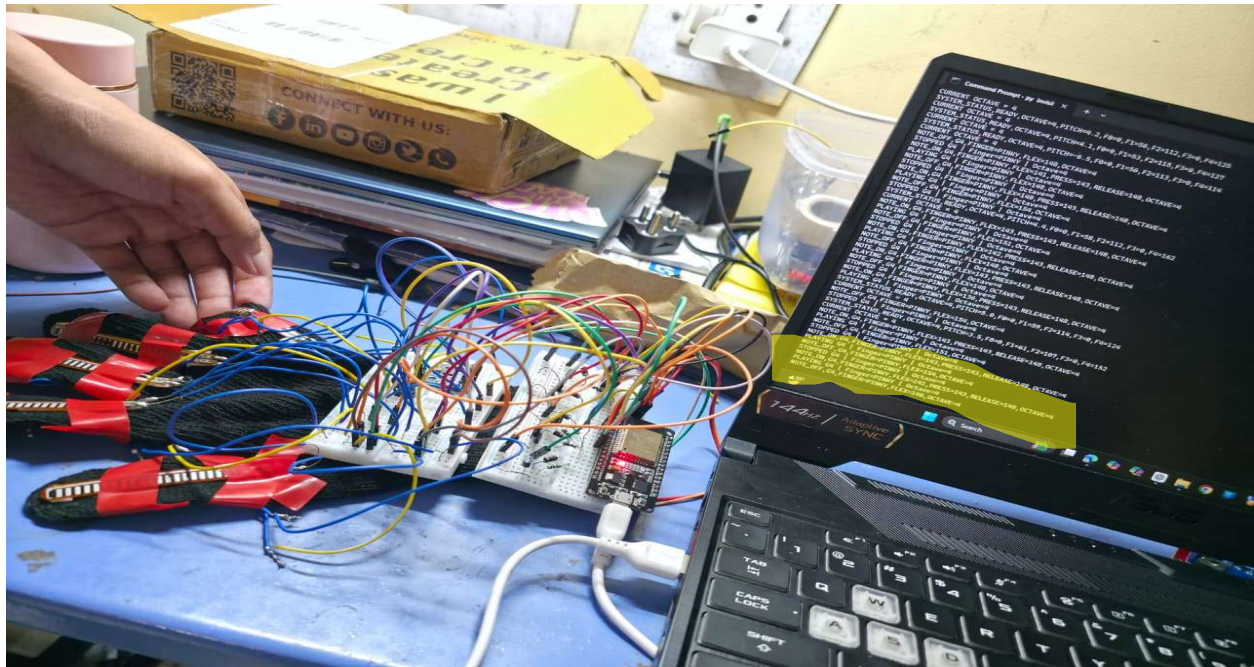
Switching between different Octaves:

```
Command Prompt - py invisib
STOPPED D4 | Finger=INDEX | Octave=4
NOTE_ON, D4, FINGER=INDEX, FLEX=2665, PRESS=3394, RELEASE=3597, OCTAVE=4
PLAYING D4 | Finger=INDEX | Octave=4
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=-15.5, F0=0, F1=49, F2=109, F3=0, F4=147
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-74.0, F0=0, F1=48, F2=110, F3=0, F4=135
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-77.7, F0=0, F1=48, F2=109, F3=0, F4=137
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-69.1, F0=0, F1=73, F2=109, F3=0, F4=137
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-69.6, F0=0, F1=47, F2=108, F3=0, F4=138
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-72.8, F0=0, F1=48, F2=101, F3=0, F4=138
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-62.5, F0=0, F1=52, F2=88, F3=0, F4=140
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-67.2, F0=0, F1=52, F2=104, F3=0, F4=140
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-67.6, F0=0, F1=53, F2=108, F3=0, F4=139
CURRENT OCTAVE = 5
NOTE_OFF, G5, FINGER=PINKY, FLEX=149, OCTAVE=5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-68.0, F0=0, F1=53, F2=110, F3=0, F4=158
CURRENT OCTAVE = 5
NOTE_ON, G5, FINGER=PINKY, FLEX=140, PRESS=143, RELEASE=148, OCTAVE=5
PLAYING G5 | Finger=PINKY | Octave=5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-67.2, F0=0, F1=54, F2=110, F3=0, F4=135
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-57.8, F0=0, F1=57, F2=111, F3=0, F4=137
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=5, PITCH=-35.2, F0=0, F1=54, F2=111, F3=0, F4=139
CURRENT OCTAVE = 5
SYSTEM_STATUS, READY, OCTAVE=4, PITCH=-21.1, F0=0, F1=57, F2=111, F3=0, F4=139
CURRENT OCTAVE = 4
SYSTEM_STATUS, READY, OCTAVE=3, PITCH=26.2, F0=0, F1=57, F2=112, F3=0, F4=141
CURRENT OCTAVE = 3
SYSTEM_STATUS, READY, OCTAVE=3, PITCH=24.3, F0=0, F1=54, F2=115, F3=0, F4=141
CURRENT OCTAVE = 3
NOTE_OFF, G3, FINGER=PINKY, FLEX=148, OCTAVE=3
NOTE_ON, G4, FINGER=PINKY, FLEX=142, PRESS=143, RELEASE=148, OCTAVE=4
PLAYING G4 | Finger=PINKY | Octave=4
NOTE_OFF, D4, FINGER=INDEX, FLEX=3734, OCTAVE=4
NOTE_ON, D3, FINGER=INDEX, FLEX=2690, PRESS=3394, RELEASE=3597, OCTAVE=3
PLAYING D3 | Finger=INDEX | Octave=3
SYSTEM_STATUS, READY, OCTAVE=3, PITCH=25.1, F0=0, F1=77, F2=112, F3=0, F4=142
CURRENT OCTAVE = 3
SYSTEM_STATUS, READY, OCTAVE=3, PITCH=22.8, F0=0, F1=37, F2=114, F3=0, F4=141
CURRENT OCTAVE = 3
SYSTEM_STATUS, READY, OCTAVE=3, PITCH=34.4, F0=0, F1=45, F2=76, F3=0, F4=142
CURRENT OCTAVE = 3
```

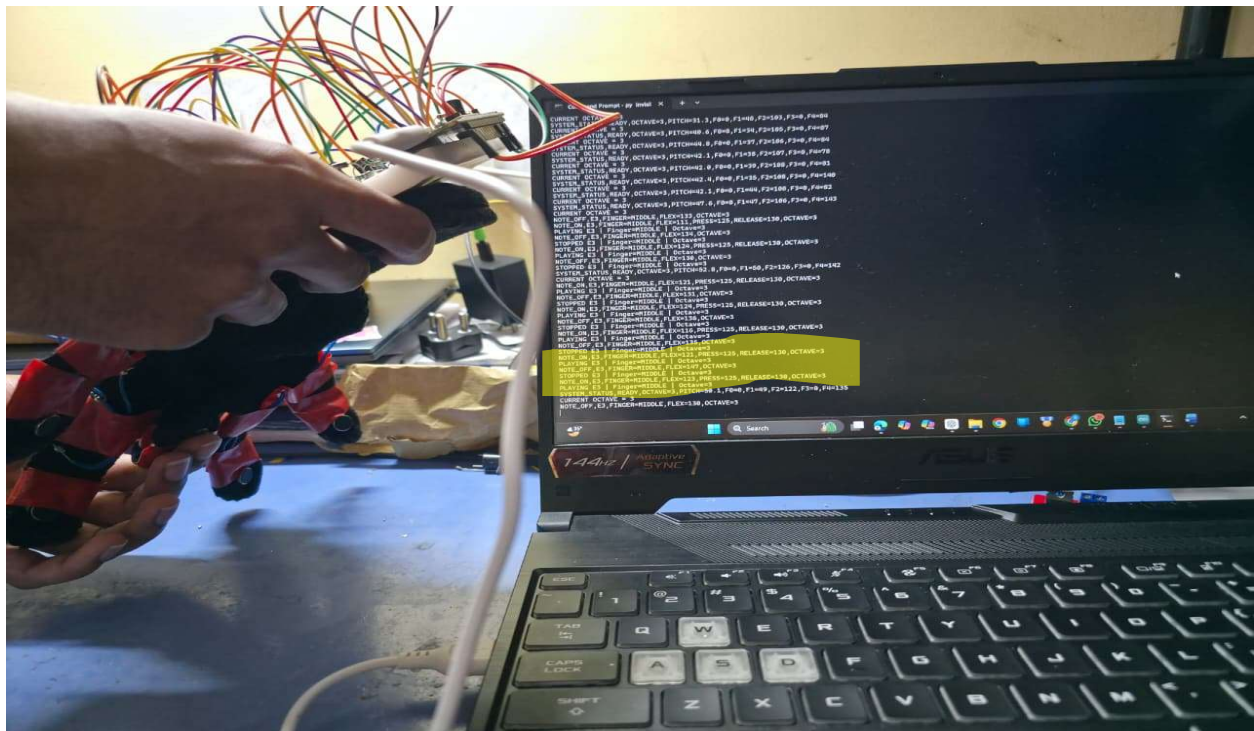
Hardware:



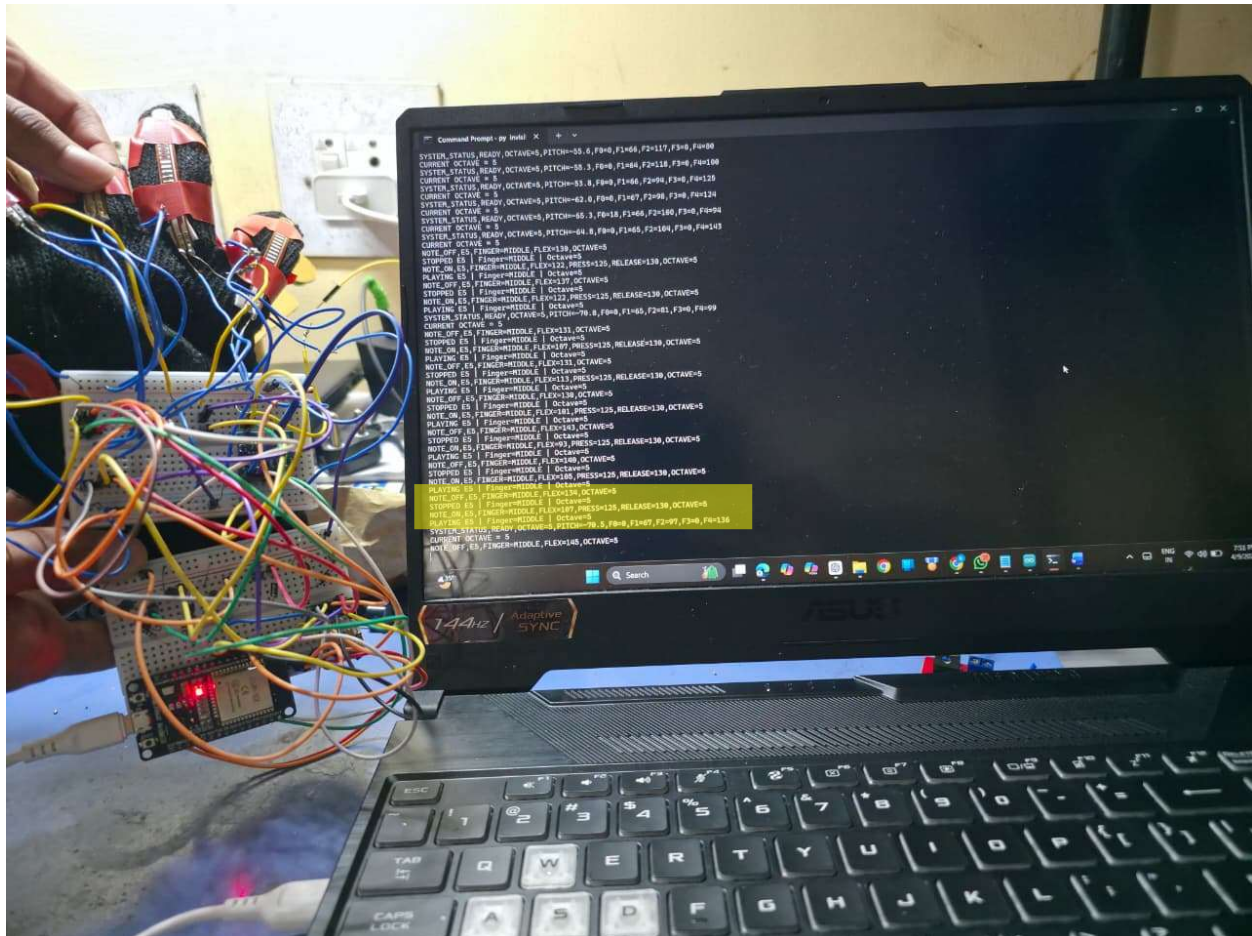
Octave 4(hand placement is 0 i.e is straight and pinky finger is bent)



Octave 3(hand placement is -90 i.e is bent down and middle finger is bent)



Octave 5(hand placement is 90 i.e is lifted up and middle finger is bent)



Conclusion

The Invisible Piano Glove project successfully demonstrates an innovative human-computer interface using embedded systems technology. By leveraging ESP32's analog-to-digital conversion capabilities and PWM control, the system accurately detects five-finger bending gestures through flex sensors and provides real-time haptic feedback through vibration motors mapped to piano notes (C4, D4, E4, F4, G4).

The implementation showcases efficient sensor calibration, debouncing logic to prevent false triggers, and proportional motor intensity based on finger bend degree. Serial monitoring provides comprehensive debugging data including raw flex values, bend percentages, and PWM

outputs. This project validates the feasibility of wearable gesture-based musical interfaces using affordable components suitable for educational demonstrations and prototype development.

The system achieves responsive real-time performance with 25ms loop timing, making it suitable for musical applications requiring precise timing. Hardware integration proves robust with proper power management separating high-current motors from ESP32's USB power. This work establishes a solid foundation for advanced gesture-controlled musical instruments and haptic feedback systems.

Future Works

1. **Full Octave Support:** Expand to 12+ fingers/keys using dual gloves or hand position tracking with MPU6050/IMU for complete piano keyboard emulation.
2. **Gesture Recognition:** Add wrist IMU sensors for additional control gestures (volume, octave shift, sustain pedal emulation).
3. **Wearability Enhancement:** 3D-print custom glove housing with flexible PCB integration for production-ready form factor.
4. **AR/VR Integration:** Stream finger positions to Unity/Unreal Engine for mixed-reality piano visualization and multiplayer performance.

References:

Air Piano Interaction via Flex Sensors and Computer Vision on Edge Devices.

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10574644>

DOI- [10.1109/AIIoT58432.2024.10574644](https://doi.org/10.1109/AIIoT58432.2024.10574644)

Investigation of the Effect of the Location of Flex Sensors in a Sensor Glove for Accurate Tracking of Hand Movements.

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/11054194>

DOI- [10.1109/USBREIT65494.2025.11054194](https://doi.org/10.1109/USBREIT65494.2025.11054194)

Smart glove with gesture recognition ability for the hearing and speech impaired

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/6967567>

DOI-[10.1109/GHTC-SAS.2014.6967567](https://doi.org/10.1109/GHTC-SAS.2014.6967567)

Machine Learning Enhanced Smart Sensors for Hand Gesture Recognition

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10824205>

DOI-[10.1109/ISSTC63573.2024.10824205](https://doi.org/10.1109/ISSTC63573.2024.10824205)

Design of Multichannel Tactile Feedback Digital Glove for Virtual Reality

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10415177>

DOI: [10.1109/LSSENS.2024.3358260](https://doi.org/10.1109/LSSENS.2024.3358260)

A High-Performance Flexible Strain Sensor for Smart Glove in Human-Machine Interaction

<https://ieeexplore-ieee-org.egateway.vit.ac.in/document/11170032>

DOI: [10.1109/NEMS67320.2025.11170032](https://doi.org/10.1109/NEMS67320.2025.11170032)